第 5 章練習：行程同步

先寫出採用的記憶體、排程與同步假設，再列出狀態或推導；詳解另存於獨立文件。

OS05-E01　臨界區的三項性質

某鎖演算法保證互斥，但一名持續請求進入的執行緒可能被已結束臨界區、再次請求的其他執行緒無限次超越。

1. 這違反互斥、進度（progress）、有限等待（bounded waiting）中的哪項性質？
2. 此情況是否必然是 deadlock？說明 starvation 的可能情境。
3. 假設共有固定的 n 名參與者，各自最多有一個未完成請求；取得 ticket 的動作是原子的，票號不繞回。若採 FIFO ticket，前任最終離開、等待者最終獲排程，從 Pi 取得票號起算，最多有多少次他人進入在 Pi 前面？

OS05-E02　有界緩衝區的鎖定順序

容量為 3 的緩衝區由 producer、consumer 使用 `mutex`、`empty`、`full` 保護。現在三格都已放入完整項目，`empty=0`、`full=3`，`mutex` 可用；無其他操作正在保留槽位。

1. 正確的 producer 在進入資料結構臨界區前，應先等待哪個 semaphore？
2. Consumer 成功取出一項後，給出一種正確的釋放 `mutex` 與通知空槽的順序。
3. 若 producer 先持有 `mutex`，再因 `empty=0` 阻塞，consumer 為何無法完成取出？假設兩者均依序等待題述 semaphore。

OS05-E03　條件變數與狀態條件

等待者使用 `if (!ready) wait(cv); use(buffer);`；通知者設定 `ready=true` 後呼叫 `signal(cv)`。

1. 為何只用一次 `if` 可能不安全？
2. 用一把共同 mutex 改寫檢查、等待與使用緩衝區的模式，說明每次返回後應做什麼。
3. 若通知先於等待者檢查發生，在檢查之前 `ready` 一直保持 true，等待者會如何行動？若其他執行緒已將它改回 false，又應如何處理？

OS05-E04　Peterson 演算法與記憶體順序

兩個執行緒 P0、P1 使用 `atomic flag[2]` 與 `atomic turn`，初值為 `flag={false,false}`、`turn∈{0,1}`。對 i∈{0,1}、j=1−i，進入協定依序為 `flag[i]=true`、`turn=j`、`while(flag[j] && turn==j)` 等待；離開臨界區時設 `flag[i]=false`。本題所有共享原子讀寫均採 sequential consistency（SC）。討論活性時，假設相關執行緒持續獲得排程，且已進入臨界區者最終離開。

1. 以兩者進入前最後一次寫入 `turn` 的先後，證明互斥。
2. 從 Pi 完成 `flag[i]=true; turn=j` 起，在 Pi 進入之前，Pj 最多能進入幾次？說明重新進入時對 `turn` 的影響。若只從設旗標起算，為何不能直接套用這個次數界？
3. 在上述前提下，分別說明互斥、progress 與 bounded waiting 為何成立；次數上界是否就是實際時間上界？
4. 若以一般非原子物件取代所有共享原子，C/C++ 中為何不能沿用 SC 的逐步交錯證明？

OS05-E05　Semaphore 與緩衝區狀態

容量 3 的 FIFO 緩衝區初始為空，`empty=3`、`full=0`、`mutex=1`。Producer 依序放入 A、B、C；consumer 取出 A；producer 接著放入 D。各操作都按正確 semaphore 順序完整結束後，才進行下一項。

1. 列出每個操作結束後 `empty`、`full`、`mutex` 與緩衝區內容。
2. `mutex` 是初值為 1 的 binary semaphore 時，為何第一個持有者尚未釋放前，第二個等待者不能同時進入？另以一種「負數表示等待者、signal 原子交棒許可」的具體實作，追蹤兩個 wait 與兩個 signal；不要把該計數表示法當成所有 semaphore 的共同規定。
3. 若緩衝區再次全滿時，producer 先取得 `mutex` 才在 `empty` 上阻塞，consumer 能否完成取出？畫出兩者的等待關係。
4. 一般執行中，為何不能隨時只用 `empty+full=3` 檢查？請定義並計入尚未發布的在途槽位。

OS05-E06　Mesa、Hoare 與遺失喚醒

監視器的 mutex 保護緩衝區及初始為 false 的 `ready`。等待者持鎖執行 `while(!ready) wait(cv)`；通知者持同一把鎖設 `ready=true` 後 signal。

1. 為何釋放 monitor 並提交等待必須相對於通知者取得鎖及通知的行為形成原子交接？
2. 若 signal 先於等待者檢查發生，且 `ready` 在檢查時仍為 true，等待者會如何行動？若另一執行緒已將它改回 false，為何重新等待不等於遺失喚醒？
3. Mesa 語意下，若通知後另一執行緒先把 `ready` 改回 false，已喚醒者重取鎖後能否直接使用 buffer？
4. 比較 Hoare signal-and-wait 與 Mesa signal-and-continue；假設 Hoare 通知時 predicate 已為真、立即交接、無虛假喚醒或逾時。這些前提和排程交接成本有何影響？

OS05-E07　被通知者沒有拿到唯一項目

採 Mesa 語意，使用一把 mutex `m` 和 condition variable `cv`；佇列 `q` 初始為空。C1、C2 各執行 `lock(m); while(q.empty()) wait(cv,m); pop(q); unlock(m)`；P 只執行一次 `lock(m); push(q,A); signal(cv); unlock(m)`。此例沒有虛假喚醒，`m` 不保證 FIFO。

依序觀察：T0：C1 wait；T1：P 持鎖放入 A；T2：P 仍持鎖並通知 C1；T3：P 釋鎖後，C2 完整取走 A 並釋鎖；T4：C1 重取鎖；T5：C1 採取下一步。

1. 對 T0 至 T5 各事件結束後，列出 `m` 的持有者、`q`、CV 等待者，以及「已通知但尚未重取鎖」集合 E。
2. 若把 `while` 改成 `if`，第一個錯誤操作在哪一步？為何這不是虛假喚醒或遺失喚醒？
3. 若自行拆成先 unlock、稍後才登記等待，列出使 q 非空但 C1 睡著的三步交錯。這是否符合 `pthread_cond_wait` 的呼叫契約？
4. 若 C1、C2 都已等待，P 放入一個 A 後改用 broadcast，兩人能否各取一項？`while` 是否保證公平或最終完成？

OS05-E08　Bakery 選號與同號

三位參與者執行本章的 Bakery 協議，共享變數為順序一致的原子物件，票號不溢位，所有人最初尚未取號。

1. P0、P1 均在發布票號前看到最大值 0，並各選號 1。比較兩者的優先順序；票號為何可相同？
2. 若略過等待 `choosing[j]`，構造兩人同時進入臨界區的交錯。
3. 在每人最多一個未完成請求等前提下，從 Pi 完成取號起可有多少其他現存或正在選號的請求先進入？要推得 Pi 最終進入還需哪些活性假設？

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 5. Lamport, A New Solution of Dijkstra’s Concurrent Programming Problem, 1974. pthread_cond_wait(3p); Windows spin locks; Windows critical sections; Linux lock types; OpenMP critical.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study